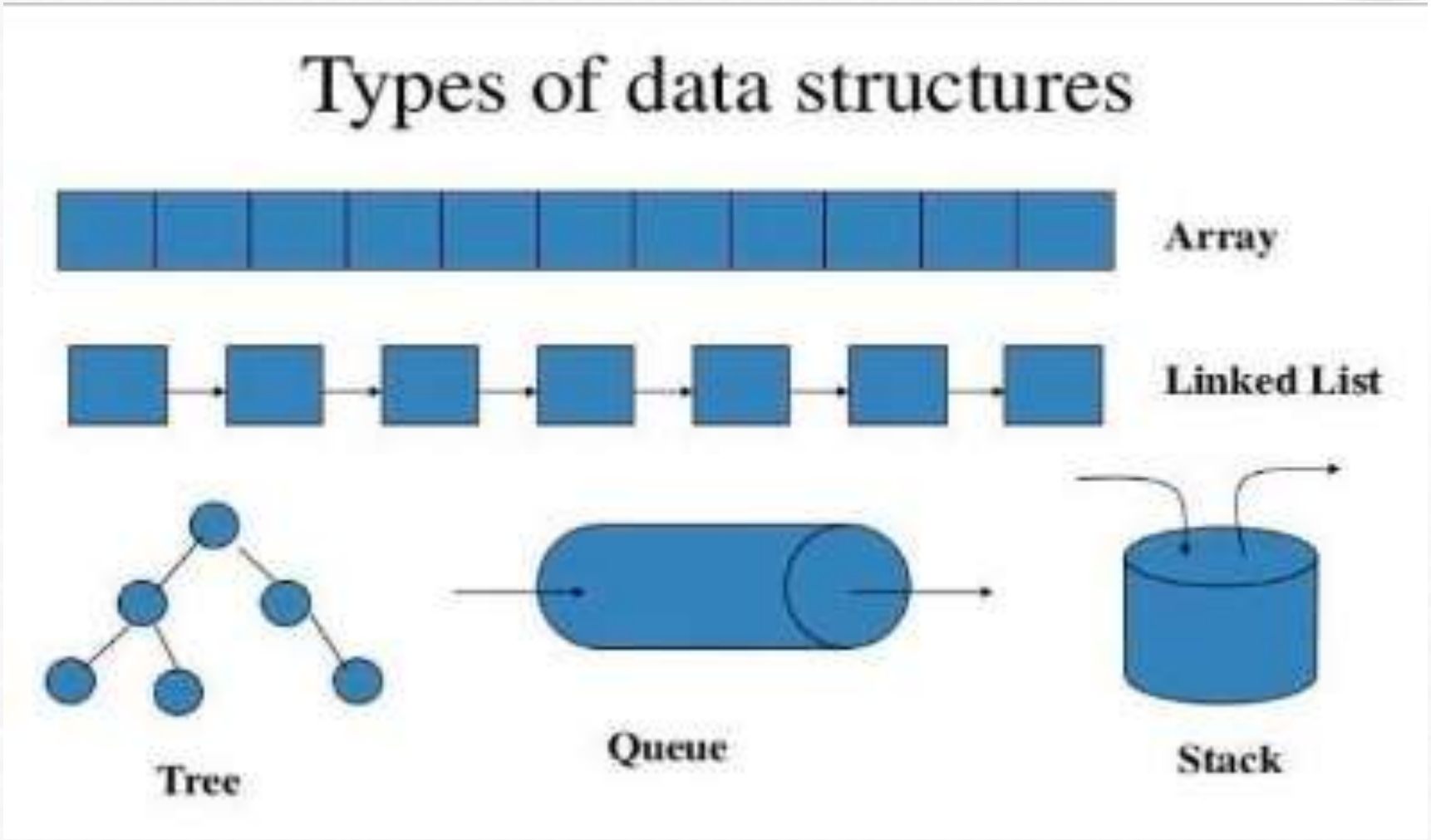
Data Structure Using C++

Lecture : 5

Queues

Dr. Ali Hamzah

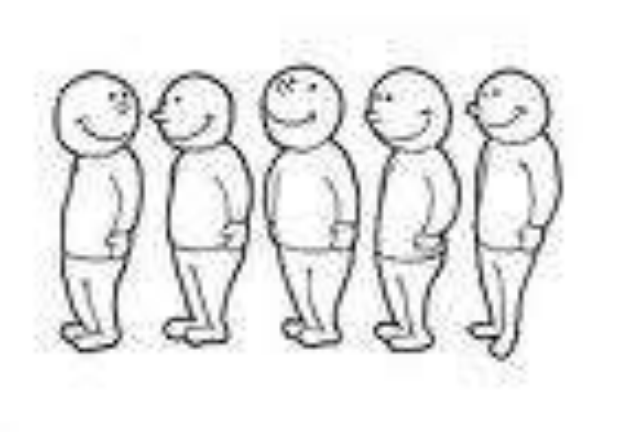
Review *Data Structures ADT*



Queues

1. A **queue** is a linear data structure that resembles a waiting line that grows by adding elements to its end and shrinks by taking elements from its front.

2. It is FIFO structure.



Queue Applications Examples

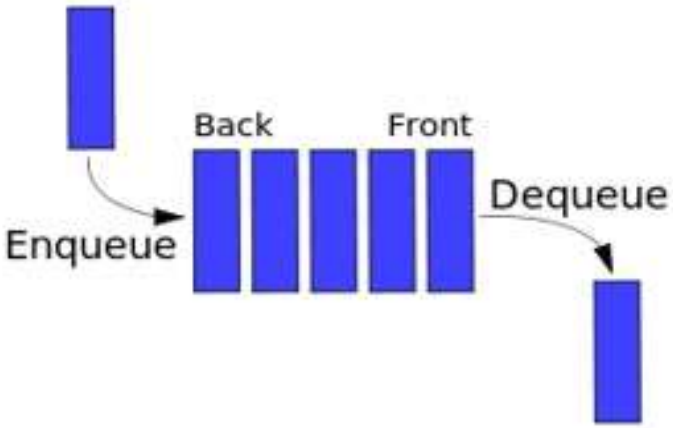
- When placed on hold for telephone operators, you may get a recording that says, "Thank you for calling A-1 Company. Your call will be answered by the next available operator. Please wait." This is a queuing system.

Queue Implementations

- What are the possible options?
  - Arrays?
  - Circular Arrays?
  - SLL?
  - DLL?
  - …….
- What are the pros and cons of each option?

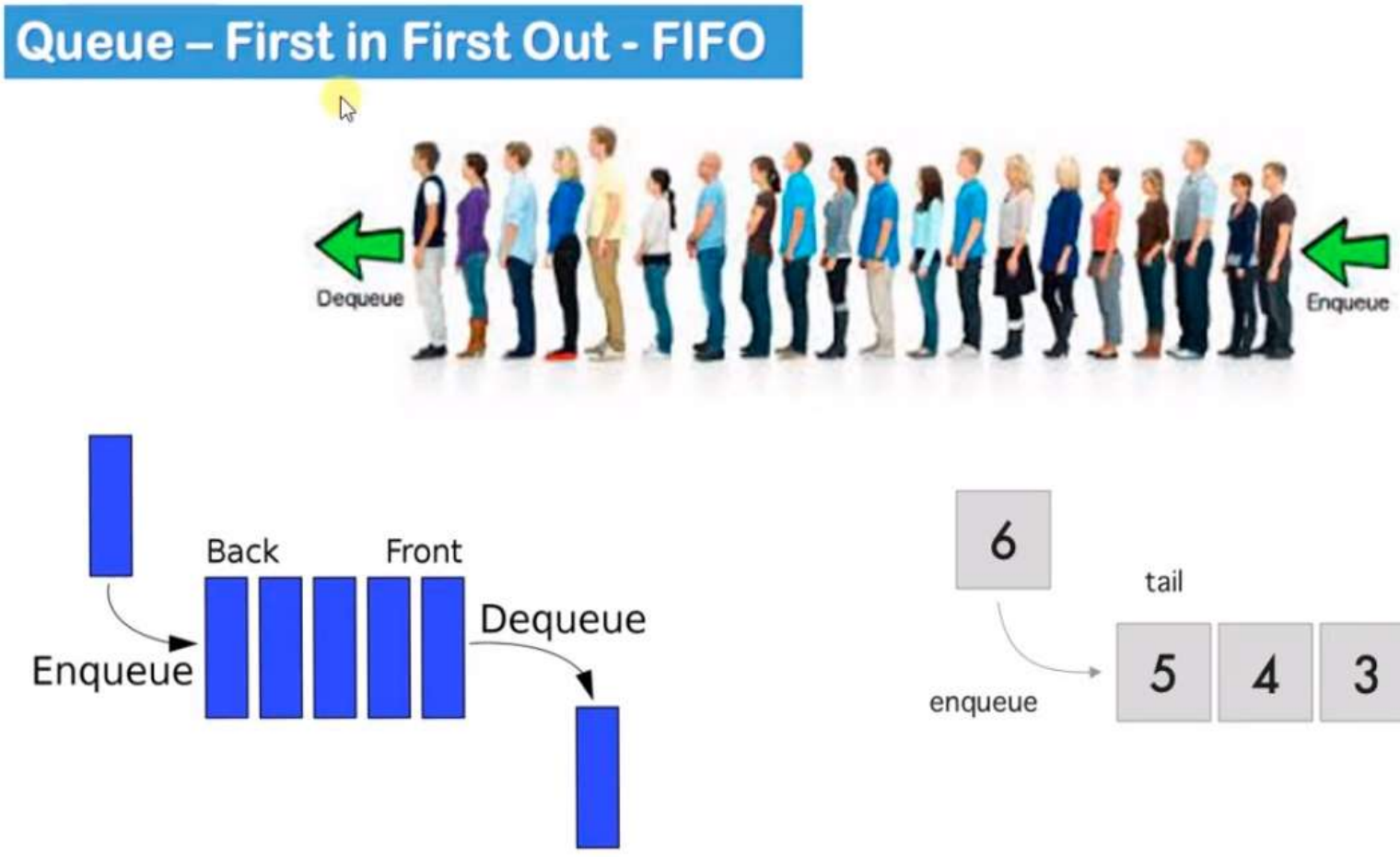
Queue Visualize

Queue Data Structure



Visualize the Queue:
https://visualgo.net/en/list

Queue Visualize (Cont'd)



Queue Baisc Operations

1. ***clear*** **()** empties the queue

2. ***isEmpty*****()** returns true if the queue is empty

3. ***enqueue*** **(*item*)** puts *item* at the end of the queue

4. ***dequeue*** **()** removes and returns item at the front of the queue

5. ***first*** **()** returns the first item in the queue without removing it

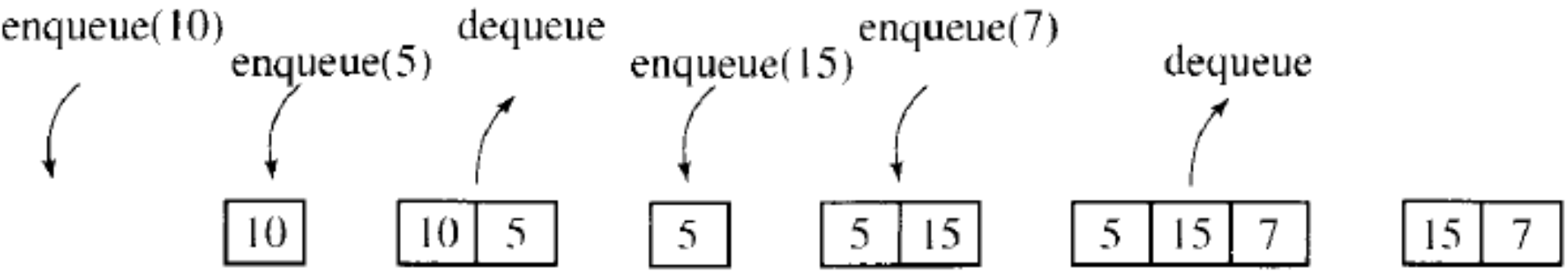
All Queue operations

Queue Operations

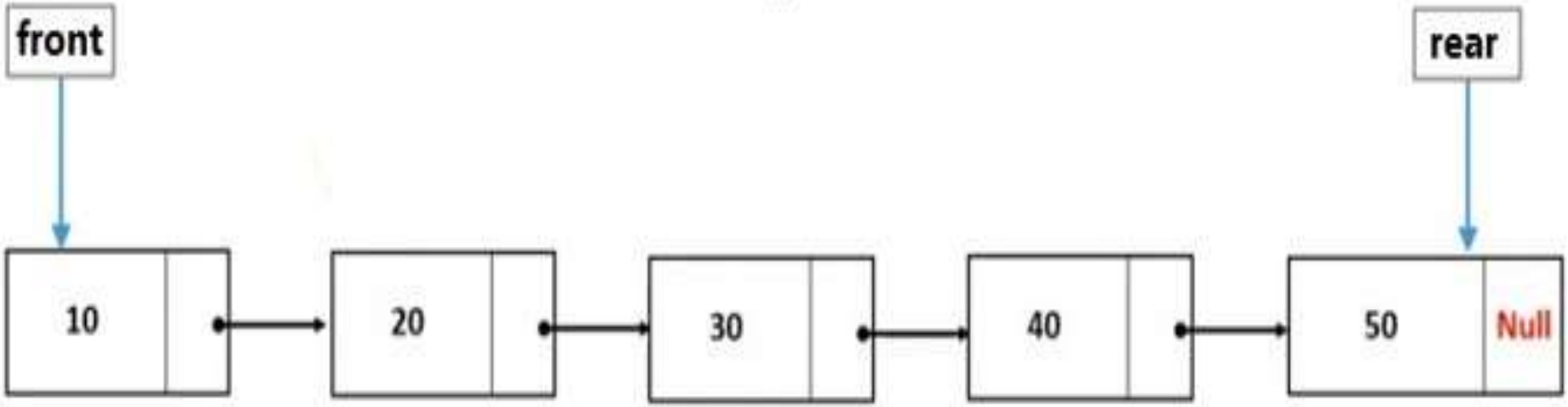
- Enqueue ( ) - Inserts an element at the end of the queue.
- Dequeue ( ) – Removes an element from the Front of the queue.
- Peek ( ) – Get the element at the Front without removing it from the queue.
- isEmpty ( ) – Check whether the queue is empty or not.
- isFull ( ) - check if there is a place to add an extra item or not.
- Clear ( ) – Remove all the elements from the queue.
- Traverse/Display () - Print all element of the queue.
- Count ( ) : return the count of queue items.
- Search ( ) : search for a specific item in the queue.

Queue Example

1. Try this sequence of operations
   - *enqueue* (10)
   - *enqueue* (5)
   - *dequeue*
   - *enqueue* (15)
   - *enqueue* (7)
   - *dequeue*



Queue using Linked List




```
Node *Qu = new node ()
Qu->data = val;
Qu->next = val;
```

```
Node * Front  = Null
Node * Rear = Null
```

Enqueue Operation

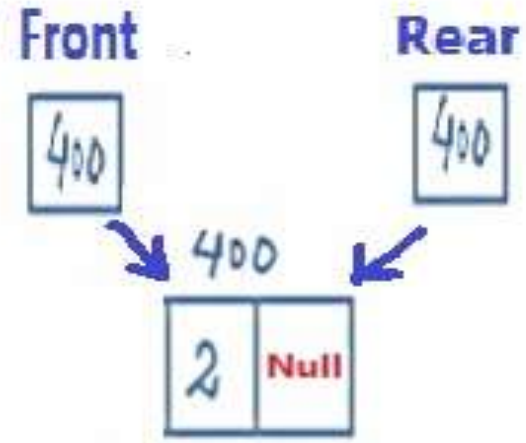

```
struct Node  {
    int data;
    struct Node* next;
  };
```

Node *Qu = new node ()
Qu->data = val;
Qu->next = val;

Node * Front = Null
Node * Rear = Null

Enqueue Operation (x)

```
struct Node  {
    int data;
    struct Node* next;
     };
```

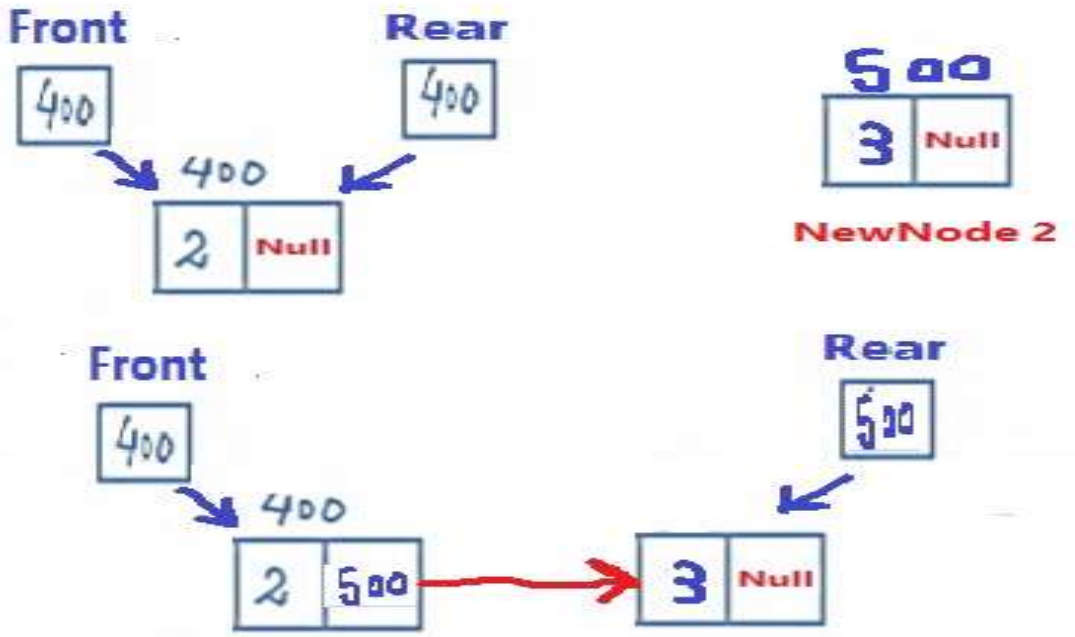



```
Node * NewNode1 = new node ()
NewNode->data = 2;
NewNode->next = Null;
```

```
Front  = NewNode
Rear = NewNode
```

Enqueue Operation (cont'd)

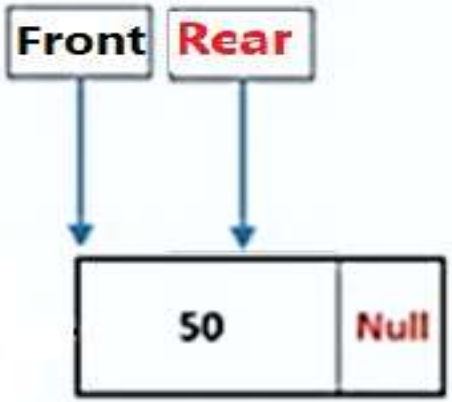
```
struct Node  {
    int data;
    struct Node* next;
     };
```



Node * NewNode2 = new node ()
NewNode->data = 3;
NewNode->next = Null;

Rear->next = NewNode
Rear = NewNode

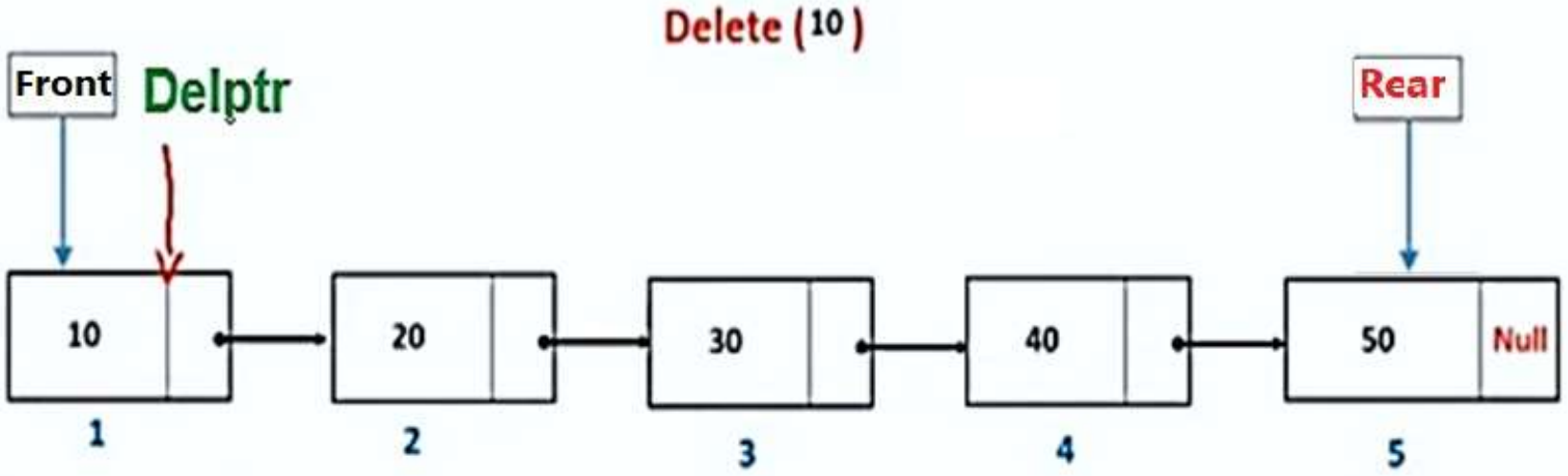
Dequeue Operation (x)



Delete Front

Front = Rear = Null

Dequeue Operation (cont'd)



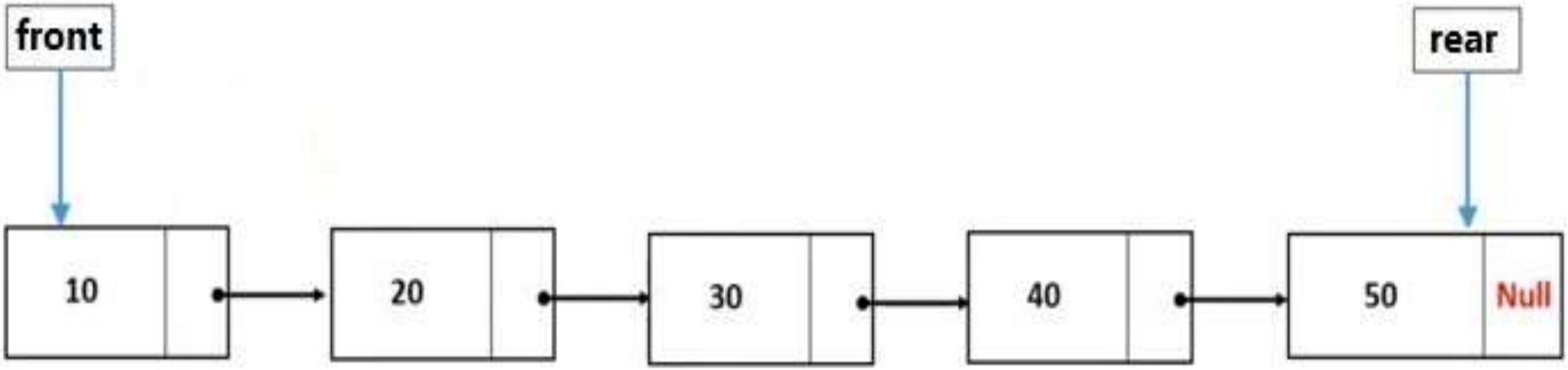
Node *Delptr

Delptr = Front

Front = Front → next;

Delete Delptr;

Peek(x)



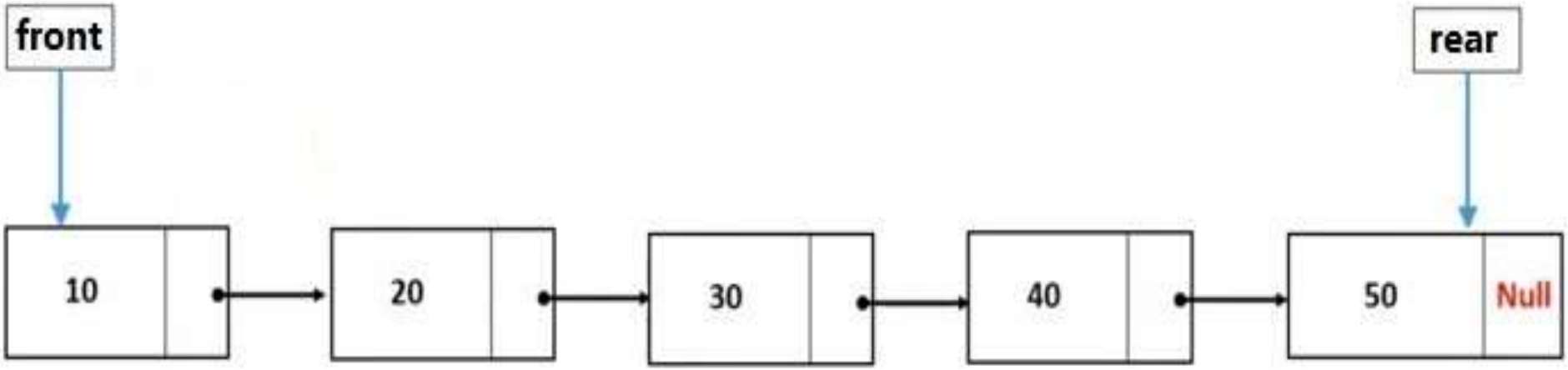
Get front

front -> Data

Get rear

rear -> Data

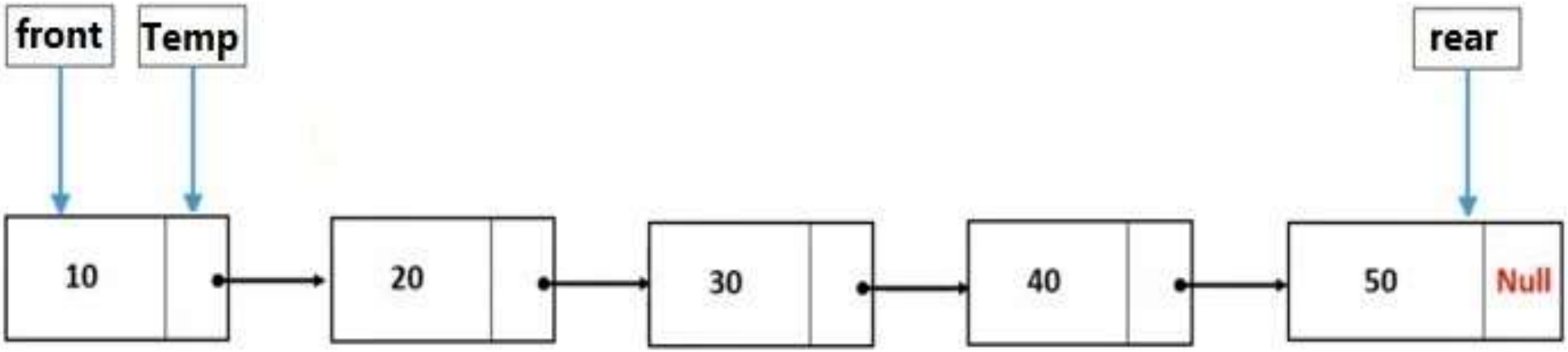
IsEmpty ()



IsEmpty ()

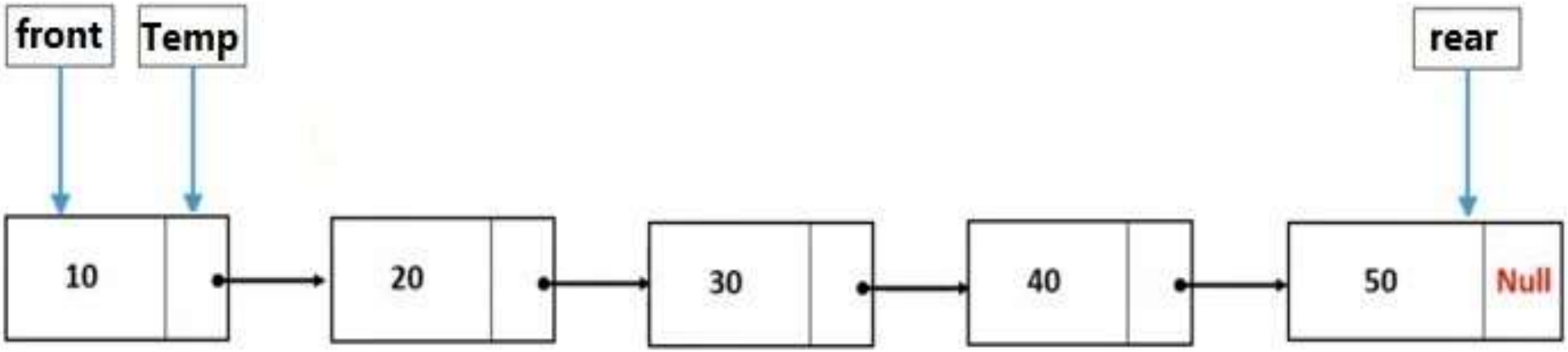
```
If (front == Null)
{
Return True
Else
Return False
}
```

Traverse



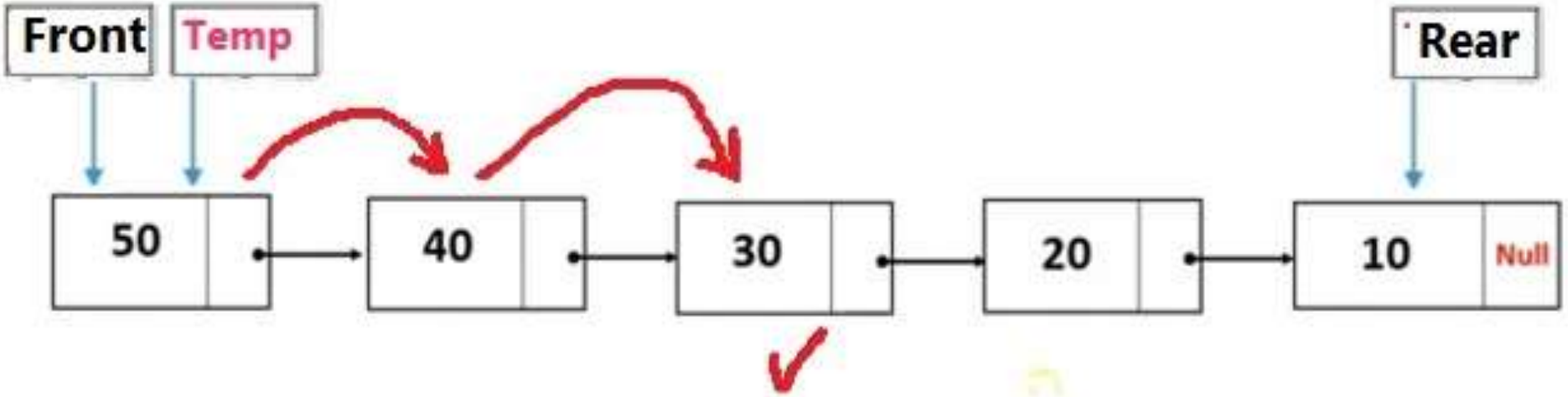

```
Node *Temp = Front;
Cout <<("List elements are - \n");
while(Temp != NULL) {
 Cout <<(" --->",Temp->data);
  Temp = Temp->next;
    }
```

Count()



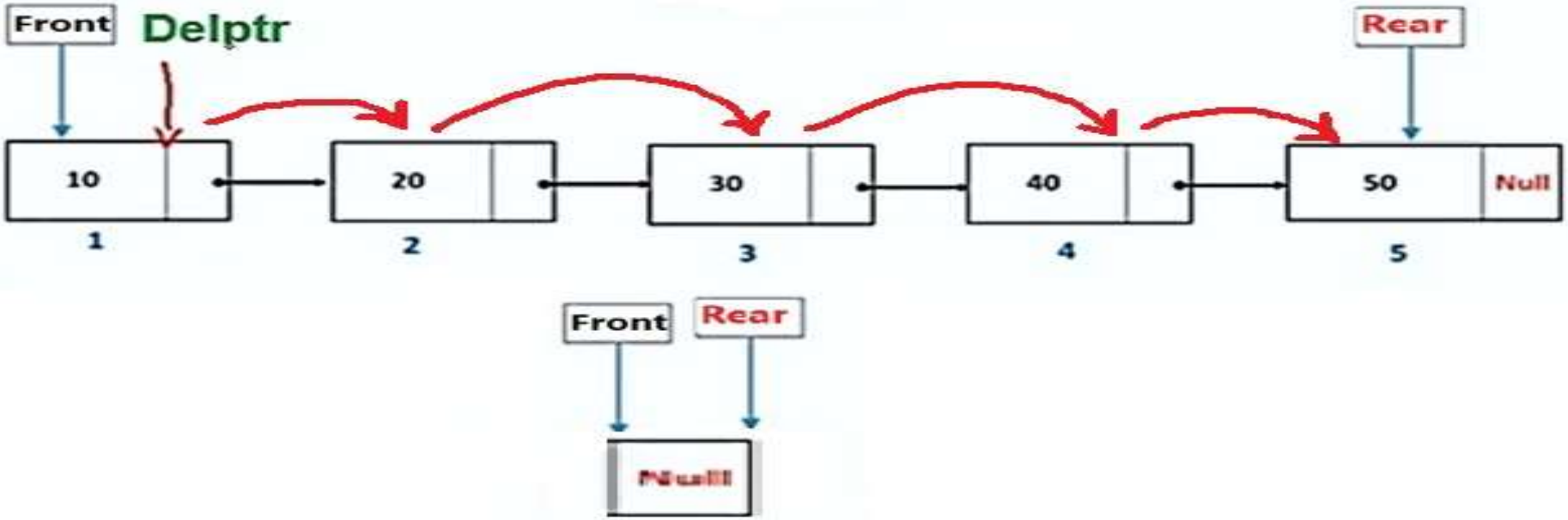

```
Int Count =0;
Node *Temp = Front;
Cout <<("List elements are - \n");
while(Temp != NULL) {
        Count ++;
      Temp = Temp->next;
        }
Return count;
```

Searching




```
Node *Temp = Front;
while(Temp != NULL)          30
 {
If (Temp->Data == Item)
   {
     Cout <<" item is", Temp->data;
     }
 Else
   {
     Temp = Temp->Next;
    }
```

Clear ()




```
while(!isEmpty)
 {
   Dequeue ()
   }
```

Queue Applications

1. Queues have numerous applications in life, system modeling and computing.
2. An entire science is dedicated to studying queues; which is called Queuing Theory.
3. We are not interested in the dynamics of the queue but we take it as a data structure.

Data Structure In Real Life

Array

Tray of Glasses

Carton of eggs represent two-dimensional array

Data Structure In Real Life

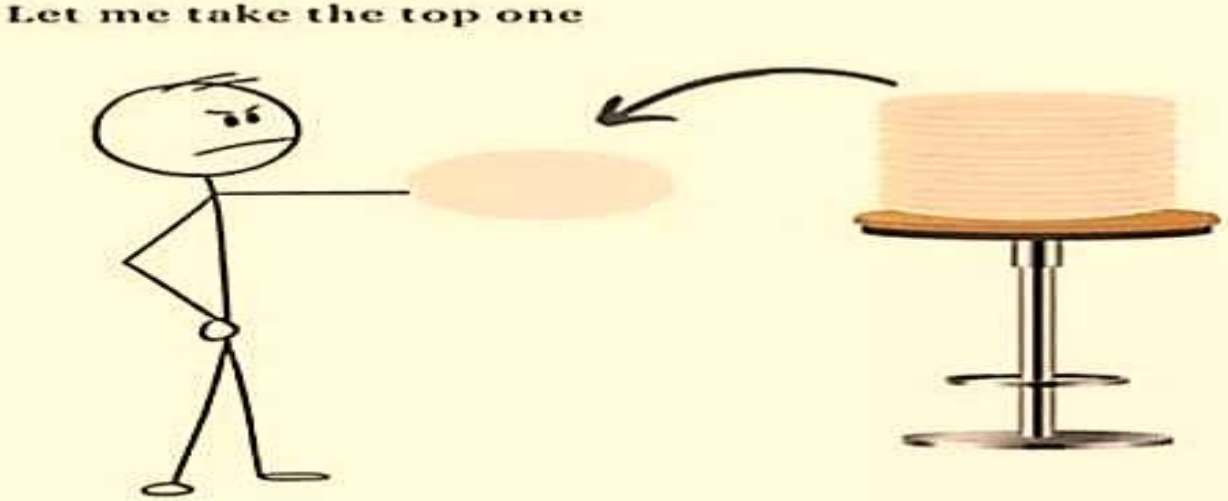
LinkedList

Music Player – Songs in the music player are linked to the previous and next songs.

Data Structure In Real Life

Stack

Pile of plates



Data Structure In Real Life

Queue

Students standing in a line in canteen